Classification

Decision trees

Affenpinscher

Brussels Griffon

Cavalier King Charles Spaniel

Chihuahua

Chinese Crested

English Toy Spaniel

Havanese

Italian Greyhound

Japanese Chin

Maltese

Manchester Terrier

Miniature Pinscher

Papillon

Pekingese

Pomeranian

Pug

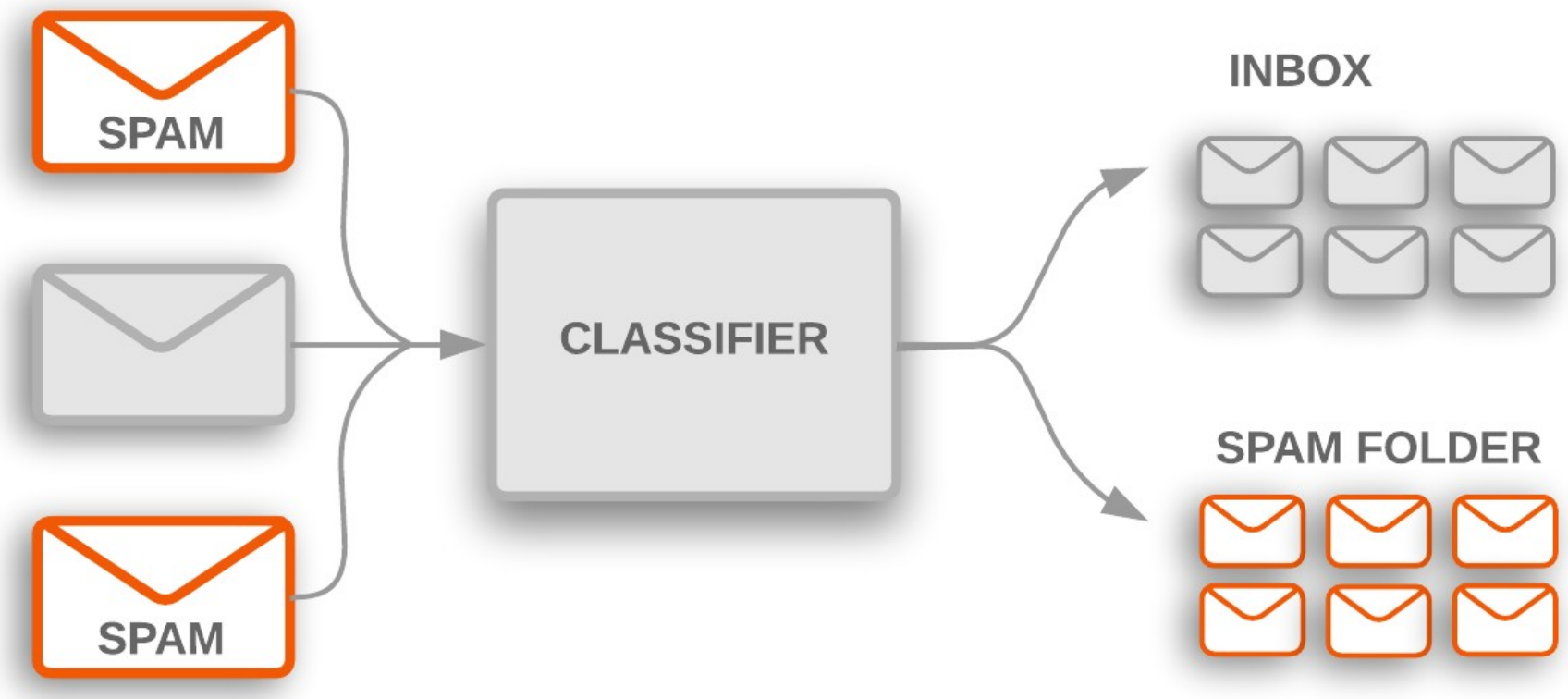
Shih Tzu

Silky Terrier

Toy Fox Terrier

Toy Poodle

Yorkshire Terrier


SPAM
SPAM
CLASSIFIER
INBOX
SPAM FOLDER

MEMBERSHIP CARD
UNIVERSITY
PASS
UCI Transportation

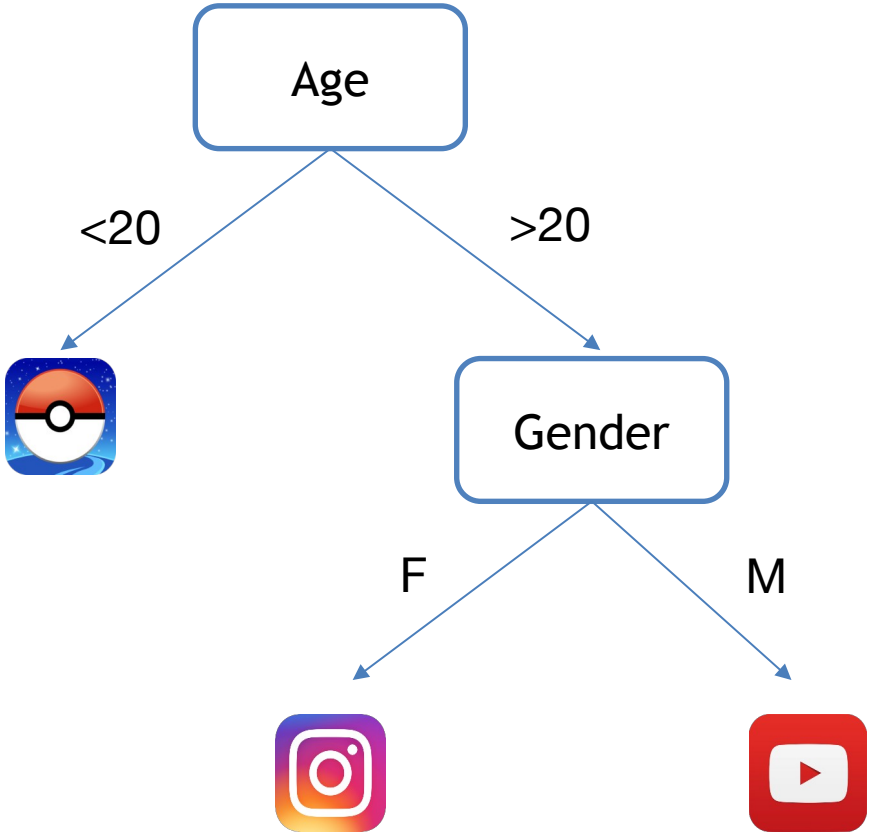
Decision trees

Application recommendation

The table opposite shows the features of different people and the applications installed by them.

What would you recommend to a new user?

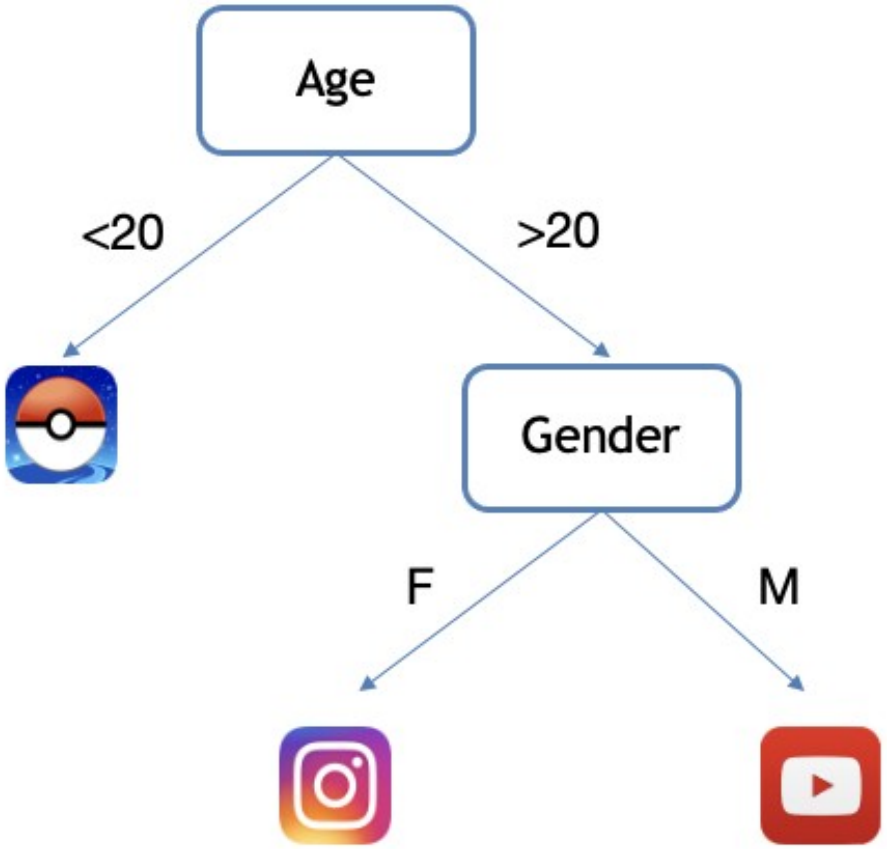
Gender	Age	Application
K	15	Pokemon Go
K	25	Instagram
M	32	YouTube
K	40	Instagram
M	12	Pokemon Go
M	14	Pokemon Go



Gender	Age	Application
K	15	Pokemon Go
K	25	Instagram
M	32	YouTube
K	40	Instagram
M	12	Pokemon Go
M	14	Pokemon Go

The computer will ask a different question...

Which characteristic: gender or age? more strongly determines the application a user will install?

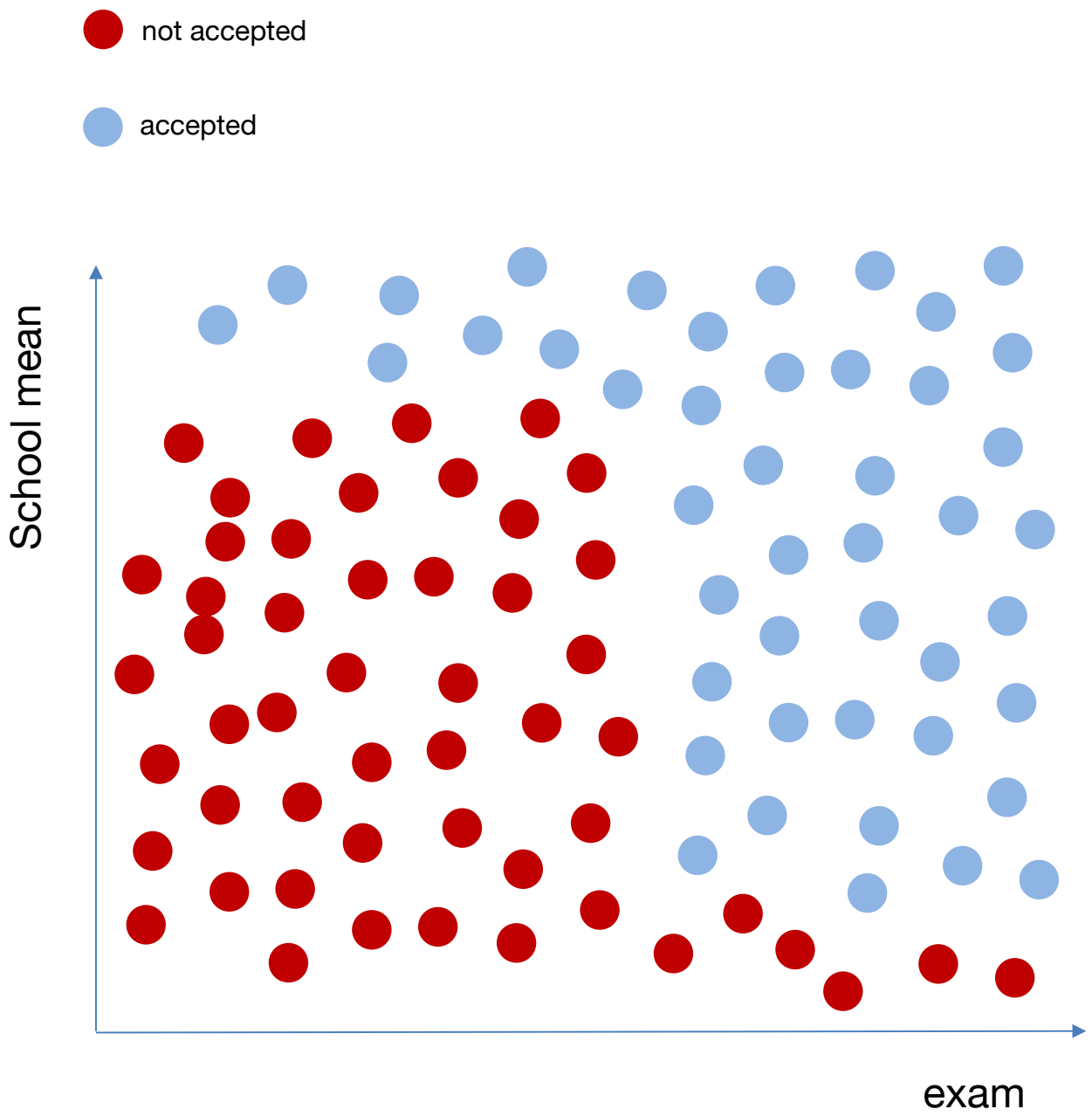


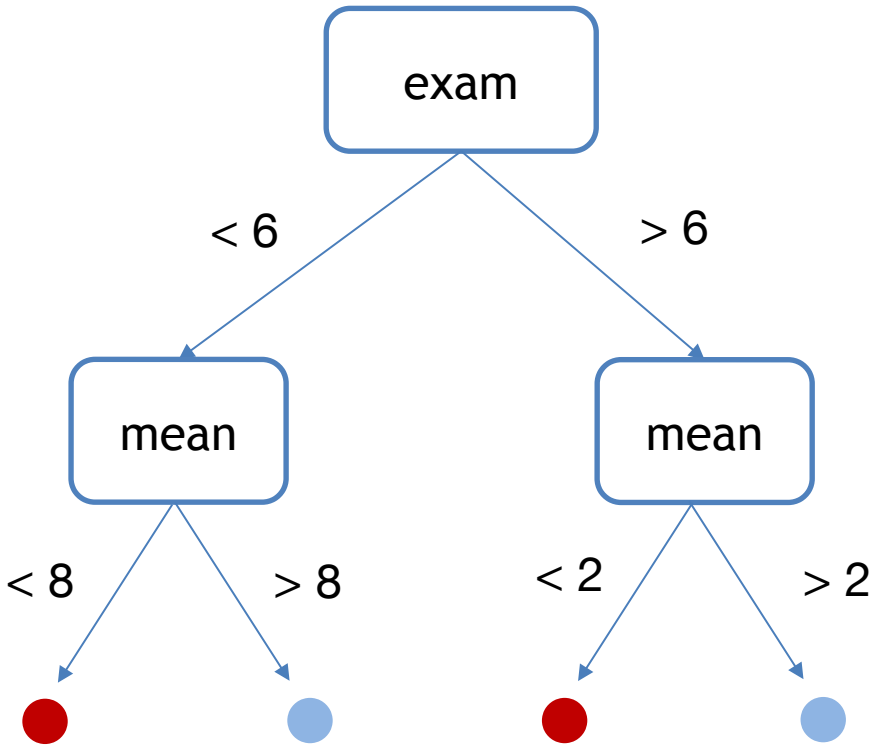
This also works for continuous variables....

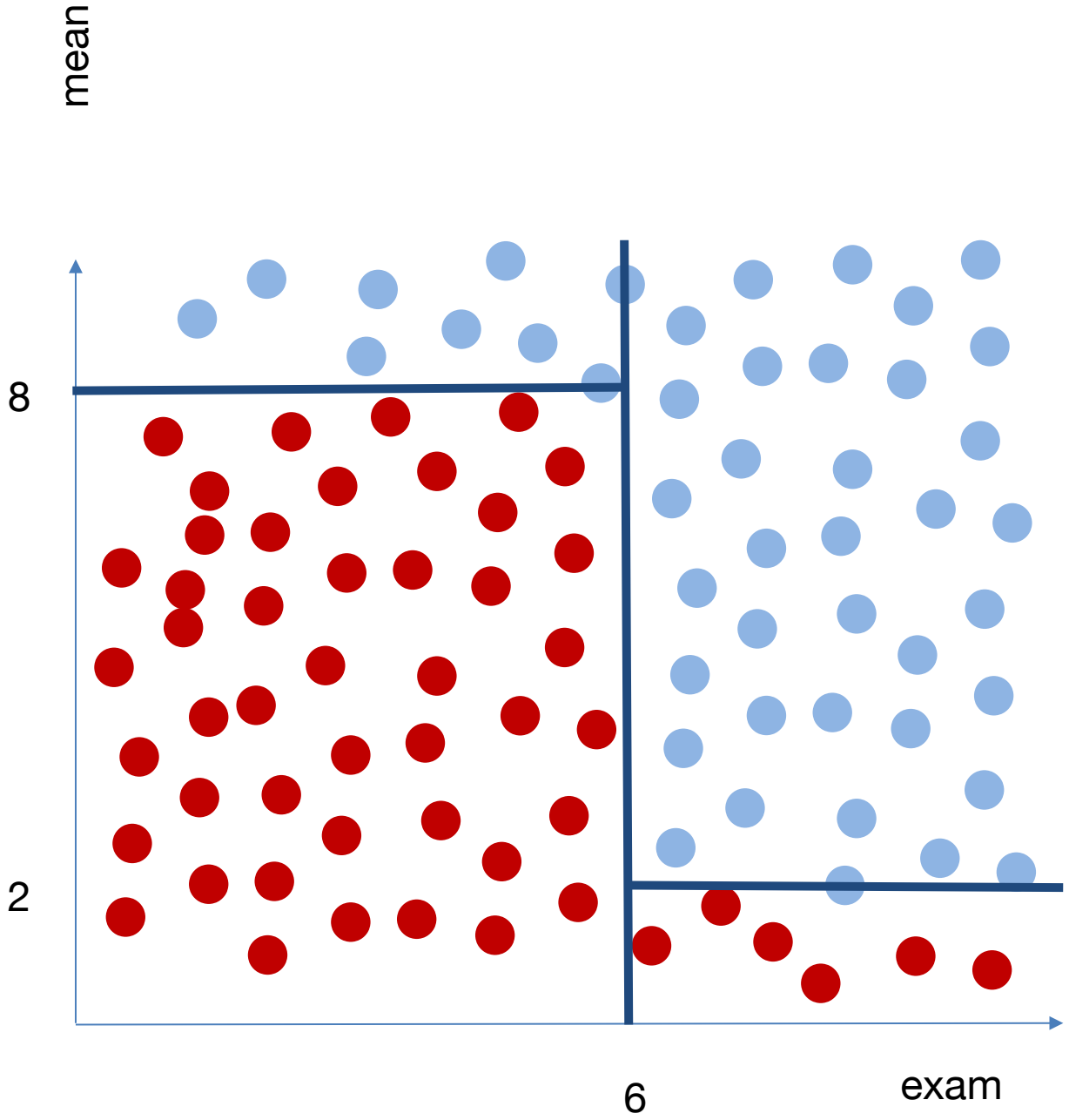
Which characteristic better predicts college admission: grade point average or high school graduation exam scores?

The question can be asked differently:

Which line better divides the points: vertical or horizontal?




exam
< 6
> 6
mean
mean
< 8
> 8
< 2
> 2

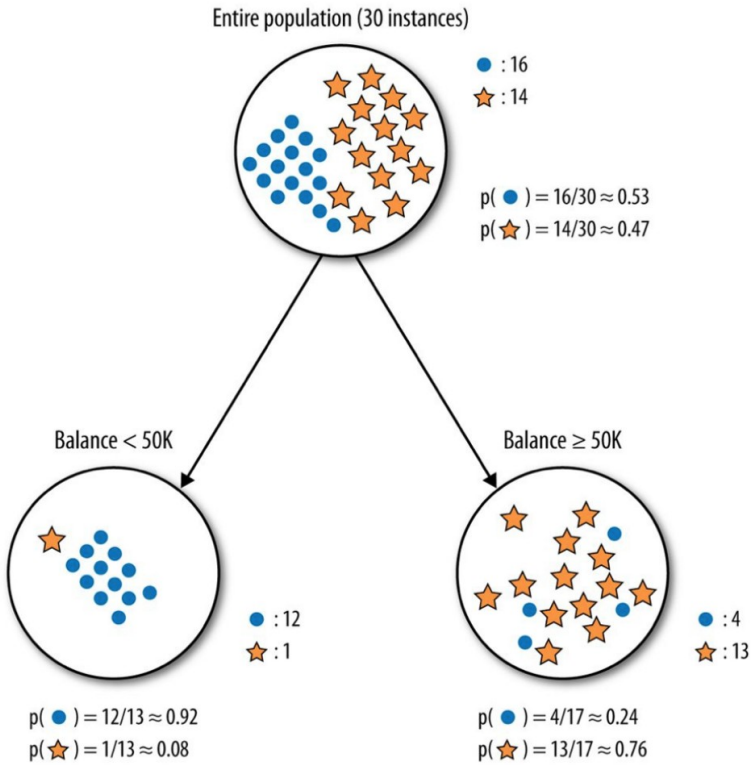

mean
8
2
6
exam

OK. But how to find these rules?

Entropy: How Decision Trees Make Decisions

The simple logic and math behind a very effective machine learning algorithm

Sam T Jan 11, 2019 · 9 min read



Provost, Foster; Fawcett, Tom. Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking

https://towardsdatascience.com/entropy-how-decision-trees-make-decisions-2946b9c18c8

I want to understand the idea behind Decision Trees classification algorithm. Create a simple dataset, with 10 rows and 3 columns, out of which at least one will be categorical. Then explain the steps the system will make in order to learn the classification algorithm.

Sure! Let's create a simple dataset with 10 rows and 3 columns. We will consider a dataset that classifies whether a person is likely to play tennis based on the weather and temperature. The dataset looks like this:

Weather	Temperature	Play Tennis
Sunny	85	No
Sunny	80	No
Overcast	83	Yes
Rainy	70	Yes
Rainy	68	Yes
Rainy	65	No
Overcast	64	Yes
Sunny	72	No
Sunny	69	Yes
Rainy	75	Yes

Now, let's explain the steps a Decision Tree classification algorithm would take to learn from this dataset:

1. **Feature selection**: The first step is to determine the best feature (column) to split the data on. This is typically done using a metric like Information Gain, which is based on entropy. In our case, we have two features: Weather and Temperature. The algorithm will calculate the Information Gain for both features and choose the one with the highest value. Let's say it chooses "Weather" as the best feature to split on.
2. **Tree construction**: The decision tree is built starting from the root node, where the dataset is divided based on the best feature ("Weather" in this case). The tree will have branches for each unique value of the chosen feature, so we'll have three branches: Sunny, Overcast, and Rainy.
3. **Splitting**: For each branch, the algorithm will again look for the best feature to split the data on, but this time it will consider only the subset of data corresponding to that branch. If there's no remaining feature to split on, or if all the data points within the branch have the same class (Play Tennis = Yes or No), the branch becomes a leaf node and is assigned the most common class in that branch.
   For the "Overcast" branch, all instances have Play Tennis = Yes, so it becomes a leaf node with the class "Yes."
4. **Recursion**: Steps 1-3 are repeated for the other branches (Sunny and Rainy) until all branches end in leaf nodes.
5. **Pruning**: After the tree is fully grown, it might be overfitting the data. To avoid this, we can prune the tree by removing branches that don't provide significant improvement to the model's performance.

After constructing the decision tree, you can use it to classify new instances by traversing the tree from the root node to a leaf node, following the branches that match the instance's feature values. The class assigned to the leaf node will be the predicted class for the new instance.

OK. How the entropy is calculated here?

Entropy is a measure of impurity or randomness in a dataset. In the context of decision trees, it's used to quantify the uncertainty in the target variable (Play Tennis in our example). Lower entropy indicates less uncertainty, while higher entropy indicates more uncertainty.

To calculate entropy, we need to calculate the proportions of each class (Play Tennis: Yes and No) in the dataset or subset and then use the following formula:

$$\text{Entropy}(S) = -\Sigma\ (p(i) * \log2(p(i)))$$

where S is the dataset or subset, p(i) is the proportion of class i in the dataset, and the summation is over all classes.

Let's calculate the entropy for our dataset:

1. Calculate the proportion of each class:

Play Tennis = Yes: 6/10 = 0.6
Play Tennis = No: 4/10 = 0.4

2. Apply the entropy formula:

$$\text{Entropy}(S) = -(0.6 * \log2(0.6) + 0.4 * \log2(0.4)) \approx 0.971$$

Now let's calculate the entropy for a subset of our dataset. For example, when Weather is Sunny:

Weather	Temperature	Play Tennis
Sunny	85	No
Sunny	80	No
Sunny	72	No
Sunny	69	Yes

1. Calculate the proportion of each class:

Play Tennis = Yes: 1/4 = 0.25
Play Tennis = No: 3/4 = 0.75

2. Apply the entropy formula:

Entropy(S_Sunny) = -(0.25 * log2(0.25) + 0.75 * log2(0.75)) ≈ 0.811

Calculating entropy is a key step in determining the best feature to split the data on, as it helps us compute Information Gain, which is the difference in entropy before and after splitting the dataset based on a particular feature. The feature with the highest Information Gain is considered the best feature for splitting the data.

How to deduce the information gain from this calculation?

Information Gain is used to determine the best feature to split the data on. It's calculated as the difference between the entropy of the parent dataset (before splitting) and the weighted average of the entropies of the child datasets (after splitting). The feature with the highest Information Gain is considered the best for splitting the data.

Let's calculate the Information Gain for the "Weather" feature in our dataset:

1. Calculate the entropy of the parent dataset (already calculated in the previous response):

Entropy(S) ≈ 0.971

2. Calculate the entropies of the child datasets for each unique value of the "Weather" feature:

- Entropy(S_Sunny) ≈ 0.811 (already calculated in the previous response)
- Entropy(S_Overcast):

Weather	Temperature	Play Tennis
Overcast	83	Yes
Overcast	64	Yes

Entropy(S_Overcast) = 0 (since all instances have Play Tennis = Yes)

- Entropy(S_Rainy):

Weather	Temperature	Play Tennis
Rainy	70	Yes
Rainy	68	Yes
Rainy	65	No
Rainy	75	Yes

- Entropy(S_Rainy):

Weather	Temperature	Play Tennis
Rainy	70	Yes
Rainy	68	Yes
Rainy	65	No
Rainy	75	Yes

Play Tennis = Yes: 3/4 = 0.75
Play Tennis = No: 1/4 = 0.25
Entropy(S_Rainy) = -(0.75 * log2(0.75) + 0.25 * log2(0.25)) ≈ 0.811

3. Calculate the weighted average of the entropies of the child datasets:

Weighted_Entropy = (4/10) * Entropy(S_Sunny) + (2/10) * Entropy(S_Overcast) + (4/10) * Entropy(S_Rainy)
Weighted_Entropy ≈ (4/10) * 0.811 + (2/10) * 0 + (4/10) * 0.811 ≈ 0.686

4. Calculate the Information Gain for the "Weather" feature:

Information_Gain(Weather) = Entropy(S) - Weighted_Entropy
Information_Gain(Weather) ≈ 0.971 - 0.686 ≈ 0.285

Repeat this process for the other feature(s) and compare the Information Gains. The feature with the highest Information Gain will be chosen as the best feature for splitting the data. In our case, we would also calculate the Information Gain for the "Temperature" feature and compare it to the Information Gain for "Weather" to decide which one to use for splitting the data.